SOFTWARE PATCH · REFERENCE MANUAL

The Lambda-Morph Option for covtl-pipeline

Articulatory assimilation continua on tautosyllabic CC clusters, C1C2 → C1'C2: the *vtl-synth morph* subcommand — polar-target semantic, blend option, patch procedure, command-line and Python reference, validation.

covtl-lambda-patch contributors

Add-on for covtl-pipeline (vtl-synth)
Berthommier COVTL engine · VocalTractLab 2.4

GPL-3.0-or-later

v1.2.0 · October 26, 2026

Contents

1 Introduction **2**
1.1 Motivation: articulatory continua . . . 2
1.2 What the patch adds . . . 2
1.3 Provenance and lineage . . . 2

2 The lambda morphing model **3**
2.1 The lambda variation: cluster and polar targets (v1.2.0 semantic) . . . 3
2.2 The tract-blend realization (option) . . . 3
2.3 The frame-alignment contract . . . 4
2.4 Glottis policy . . . 4
2.5 Guarantees . . . 5
2.6 What the morph is not . . . 5

3 Installing and removing the patch **6**
3.1 Requirements . . . 6
3.2 Applying: `patch/apply_patch.py` . . . 6
3.3 Removing: `patch/revert_patch.py` . . . 6
3.4 Non-regression statement . . . 7

4 Command-line reference **7**
4.1 Synopsis . . . 7
4.2 A worked session . . . 7

5 Python API reference **8**
5.1 Driving a continuum from Python . . . 8
5.2 The `morph_meta.json` record . . . 9

6 The assimilation continuum, end to end **10**
6.1 The reference experiment . . . 10
6.2 Reading the figure . . . 10
6.3 Relation to the source experiment . . . 11

7 Testing and guarantees **11**

8 Limitations and extensions **12**

1 Introduction

1.1 Motivation: articulatory continua

Phonological assimilation — *clean pan* realized as *cleam pan*, the alveolar nasal [n] taking the labial place of the following stop — is standard evidence that speech perception and production are graded rather than categorical. Behavioral work such as the compensation paradigm of Gaskell and Marslen-Wilson, and more recently the study of Pouw et al. [1] pitting neural speech recognition models against human listeners on natural assimilated speech, measures how listeners and models attribute a nasal token as it slides from its canonical place of articulation toward the assimilated one. Such paradigms need *stimulus continua*: many intermediate variants between the two endpoint pronunciations, differing articulatorily while everything else (speaker, timing, voicing, prosody) is held constant.

Articulatory synthesis is the natural tool for building these continua, because the interpolation can be performed in the control space of the synthesizer itself rather than in the acoustic signal. The `covtl-pipeline` package (`vtl-synth`) synthesizes speech as a 400 Hz sequence of 19 tract + 11 glottis VocalTractLab parameters; morphing two frame-aligned sequences in that space yields variants whose articulatory distance to each endpoint is exactly known and exactly controllable. This is what the lambda-morph option provides.

1.2 What the patch adds

The **covtl-lambda-patch** is an add-on for a `covtl-pipeline` checkout. It adds three files and one small, assert-guarded insertion:

- the engine module `vtl_synth/core/morphing.py` (the target-level and blend morphing models and drivers of Section 2);
- the CLI module `vtl_synth/cli/morph_cmd.py` (the `morph` subcommand of Section 4);
- the test file `tests/test_morphing.py` (53 tests, Section 7);
- the guarded insertion in `vtl_synth/cli/main.py` that registers the new subcommand (Section 3).

The user-facing result is a single command (since v1.2.0 the target-level semantic on the tautosyllabic CC cluster):

```
vtl-synth morph --phonetic --targets klinpan -o out/P1 --label P1
# option: the two-endpoint tract blend (v1.0/v1.1 semantics)
vtl-synth morph --phonetic klinpan klimpan -o out/P1 --label P1
vtl-synth morph out/P1_canon.tract out/P1_assim.tract -o out/P1
```

For every interpolation weight λ of a grid (default: nine uniform steps from 0 to 1), the option writes a `.tract` file and a `.wav` synthesized by VocalTractLab, plus a `morph_meta.json` provenance record. In the target mode, $\lambda = 0$ reproduces the canonical cluster *bitwise*, at $\lambda = 1$ the C1 polar target *equals* the C2 target coordinates, the glottis block is bitwise constant along the continuum, and every frame outside the C1-incident arcs stays bitwise canonical.

1.3 Provenance and lineage

The mechanism is not new: it is the generalization of the lambda experiment of the nasal assimilation study — stage A of `repro/lambda_experiment.py` in the `nasal_synthesis` reproducibility kit — which used it to probe wav2vec2 along a [n] → [m] continuum in the replication of Pouw et al. [1]. The patch extracts that stage into a reusable, documented and tested engine option, following the patch discipline of the pipeline itself: the 2026-10-18 `/N/` closure patch (`patch_N.py`) established the conventions this patch extends — minimal diff, dated rationale comment in the code, assert-guarded anchors, reversible, no change to default outputs.

2 The lambda morphing model

2.1 The lambda variation: cluster and polar targets (v1.2.0 semantic)

The lambda variation applies to a **tautosyllabic CC cluster**: C1, the nasal coda, followed by C2, the onset of the same cluster. Its effect is defined on the **polar targets** of the engine's consonantal (cluster) branch: each consonant of the cluster carries its own target (ρ_C, θ_C) in the polar control plane, and the parameter λ moves the *theta of C1 toward the theta of C2* (and, as a switchable secondary channel, its rho):

$$\theta_{C1}(\lambda) \;=\; \theta_{C1} + \lambda\,\Delta\theta, \qquad \Delta\theta = \mathrm{wrap}_{(-\pi,\pi]}(\theta_{C2} - \theta_{C1})\,, \tag{1}$$

with $\lambda = 0$ returning θ_{C1} and $\lambda = 1$ returning θ_{C2} *verbatim*; optionally $\rho_{C1}(\lambda) = (1-\lambda)\rho_{C1} + \lambda\rho_{C2}$ under `--target-rho`. The option then re-runs the engine's own syllabic trajectory builder with the C1 node carrying the λ-interpolated target, and re-synthesizes: each continuum point is a genuine cluster utterance whose C1 gesture points at the λ-interpolated place. At $\lambda = 0$ the cluster target is strictly C1C2; at $\lambda = 1$ the C1 polar target has reached the C2 target coordinates — **C1'C2**, the place-assimilated cluster: the C1 gesture now *points at* the C2 target, while C2 itself never moves. This target-level definition is the ground truth of the option; the tract-parameter blend of Section 2.2 is only its motor-level realization.

Locality is *structural* in this mode. In the engine's cluster machinery the consonant targets enter only through the sub-arcs they terminate or start from; retargeting C1 therefore modifies exactly the two C1-incident sub-arcs (the arrival arc into C1 and the departure arc C1→C2). Every frame outside them is bitwise canonical for every λ — C2 and the whole context strictly invariant — and the glottis block (voicing, F0, nasality; identity-based, not target-based) is bitwise constant along the continuum. Both properties are asserted at every run (Section 2.5).

Notation: separators, and why C.C is out of scope

Engine input notation (manual of the pipeline, authoritative): `.` = intra-phrase *syllable* separator — no pause, it *splits* C.C into a boundary sequence (a different gesture structure: the engine's cluster target rules, e.g. the stop-like alveolar target of a cluster `n`, do not apply); **no separator = CC cluster**; space = 200 ms inter-word pause; `|` = 320 ms. **The lambda variation is defined on the CC case only**: dotted forms (`klin.pan`) are a different synthesis and are rejected by the target morph — they must not appear as endpoints, denominations or examples of the lambda continuum. Endpoint phrases are written dotless: `klinpan` → `klimpan`, `minbot` → `mimbot`, `Tinpakit` → `Timpakit`, `klintan` → `klimtan`, `klinkan` → `klimkan`, `mingot` → `mimgot`. (Engine SAMPA has no `c`: a phrase like `clinpan` loses the `c` silently — the SAMPA guard warns; always verify the `.polar` phoneme inventory after a phrase change.)

`morph_meta.json` records, per item, the C1/C2 polar targets and the λ-interpolated C1 target, plus the two reference windows of the cluster: the *C1 phoneme window* (the arrival arc of C1 in the `.polar` phoneme timing — e.g. frames 232–272 for the `klinpan` family, the reference window of the assimilation study) and the *C1-incident arcs window* (the maximal footprint of λ on the trajectory).

2.2 The tract-blend realization (option)

The v1.0/v1.1 semantics — kept as an option — realize the same C1C2 → C1'C2 transition at the motor level, as a frame-wise blend of two *frame-aligned* endpoint sequences.

Let $A, B \in \mathbb{R}^{T\times 30}$ be two tract sequences at 400 Hz: T frames, 30 columns (19 tract parameters, then 11 glottis parameters), as produced by the engine or read back from `.tract` files. A is the *canonical* cluster endpoint and B the *assimilated* one (e.g. `klinpan` versus `klimpan`). For a weight $\lambda \in [0, 1]$ the morph is the frame-wise, column-wise convex combination

$$X_\lambda \;=\; (1-\lambda)\,A \;+\; \lambda\,B, \qquad 0 \le \lambda \le 1, \tag{2}$$

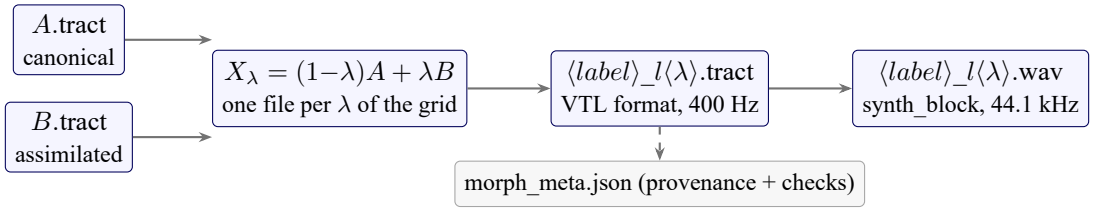


Figure 1: The lambda-morph data path. The dashed side output records source hashes, the alignment report and the endpoint identity checks of Section 2.5.

applied to *all* columns; under the default glottis policy (Section 2.4) the glottis block of A and B is required to be identical, so equation (2) acts on the tract block alone. Each λ of the requested grid produces one `.tract` file and one `.wav`:

Two properties make equation (2) the right primitive for articulatory continua. First, it is *exact at the endpoints*: in IEEE-754 arithmetic $(1-0)A+0B = A$ and $(1-1)A+1B = B$, so $\lambda = 0$ and $\lambda = 1$ do not degrade their sources (this is asserted, not assumed; see Section 2.5). Second, it is *monotone and linear in every parameter*, which gives the continuum a clean articulatory reading: at $\lambda = 0.5$ the lip-closure parameter LP is exactly halfway between its canonical and assimilated values, and the same holds for every column. Intermediate stimuli are articulatorily interpretable, not merely acoustically intermediate.

2.3 The frame-alignment contract

The morph is *spatial, never temporal*: it requires A and B to have the same number of frames T and the same column layout, and it performs no time warping. This is the frame-alignment contract. It holds by construction when the two variants are cluster minimal pairs with the same segmental skeleton — same phonemes, same durations, a single phoneme substituted — such as `klinpan` versus `klimpan`: the engine's timers are driven by segment counts and durations, which are identical, so the two synthesis runs land on the same T and the same event grid.

When the contract is violated, `check_alignment` raises `MorphAlignmentError` before anything is written. When it holds, the driver reports the *divergence window*: the range of frame indices on which the tract blocks differ. Outside that window the two sequences agree to machine precision and the blend is the identity, which is the desired behaviour: the continuum must not perturb the carrier material. (In the target mode the corresponding report is the *measured footprint* of Section 2.1, bounded by the C1-incident arcs.)

Frame alignment, operationally

Two inputs are frame-aligned when they differ by *substitution* of phonemes (ideally one), not by insertion, deletion or re-timing. Safe recipes: cluster minimal pairs in engine SAMPA (`klinpan`/`klimpan` — dotless; C.C splits are out of scope, Section 2.1); or any two `.tract` files verified with `check_alignment`. If a glottis difference is reported, decide it explicitly with the glottis policy below.

2.4 Glottis policy

The glottis block carries voicing, F0, and (via the nasal tag envelope) nasality. Blending it would confound the place-of-articulation manipulation with source changes. The option therefore makes the policy explicit:

- `common` (default) — the glottis sequences of A and B must be identical up to a tolerance (10^{-6} by default, configurable). The morph then uses them unchanged: equation (2) is a no-op on the glottis block, voicing and F0 contours are constant along the continuum, and the manipulation is a pure tract

operation. A violation raises `MorphError` with the measured maximal difference, steering the user to either fix the inputs or opt into interpolation.

- `interpolate` — the glottis block is blended with the same weight. Use it when the two variants genuinely differ in the source (for example a voiced/voiceless pair), accepting that the continuum then mixes both dimensions.

For the reference pairs the measured glottis difference is exactly 0.0: the engine derives both variants from the same segmental skeleton, so the source columns agree bitwise.

2.5 Guarantees

Every run computes and records its guarantee booleans in `morph_meta.json`. The target mode (default, v1.2.0) records:

1. `lam0_bitwise_canon` — $\lambda = 0$ reproduces the canonical cluster synthesis (`vtl-synth run <phrase>`) bitwise (`np.array_equal` on the full 30-column sequence);
2. `lam1_theta_equals_c2` — the $\lambda = 1$ item's C1 theta *equals* θ_{C2} exactly (verbatim float; `lam1_rho_equals_c2` likewise with `--target-rho`);
3. `glottis_bitwise_constant` — voicing, F0 and nasality envelopes are bitwise identical at every λ;
4. `outside_c1_arcs_bitwise_canon` — every frame outside the C1-incident arcs (padded by one 100 Hz gesture sample for the 100→400 Hz upsampling) is bitwise canonical at every λ: C2 and the context never move.

The blend mode records the v1.0/v1.1 set:

- `endpoint_lam0_equals_canon` and `endpoint_lam1_equals_assim` (or, with `confine`, `confine_lam1_equals_assim_inside` and `outside_confine_equals_canon`);
- `glottis_noop_at_lam0` (common mode).

A failed check aborts the run: in exact arithmetic these identities cannot fail (the weights are exact floats 0 and 1), so a failure would indicate a numerical anomaly worth reporting. The `.wav` of an endpoint is likewise the deterministic `synthesize_audio` of the endpoint `.tract` content — the test suite pins this byte-for-byte (Section 7).

One honest subtlety

The morph operates on the `.tract` *file representation*, whose text format quantizes parameters to six decimals. The $\lambda = 0$ output therefore reproduces the canonical `.tract` file bitwise — not the in-memory float64 sequence that the pipeline held before writing it. The difference is at most 5×10^{-7} per parameter and is a property of the file format, not of the morph.

2.6 What the morph is not

Three limitations delimit the option. First, it is *not a time-warping tool*: inputs with different segmental skeletons must be re-aligned upstream (the contract of Section 2.3). Second, the interpolation is linear *in the Maeda-type parameter space* of VocalTractLab; it is not uniformly linear in acoustic space, nor does it pass through any "natural" intermediate gesture — the intermediate states are convex combinations, and whether a given λ sounds like a good [n]/[m] half-way token is an empirical question (exactly the question the probing experiments ask). Third, the option does not synthesize endpoints itself in file mode: it consumes `.tract` files, so the speaker, language and prosody of the endpoints are whatever produced those files — which is also what makes the option speaker-agnostic.

3 Installing and removing the patch

3.1 Requirements

The patch targets a `covtl-pipeline` checkout (v1.1.0 or later, commit `1d15ed7` verified) with its Python dependencies (`numpy`; `scipy` only for the WAV convenience path). A typical installation is `pip install .` inside the checkout, as described by the pipeline's own `INSTALLATION.md`. Audio synthesis additionally requires the `vocaltractlab-cython` wheel; without it, the option still runs with `--no-audio` (and the non-VTL tests still pass). The patch scripts themselves use the standard library only.

3.2 Applying: `patch/apply_patch.py`

```
python patch/apply_patch.py [--pipeline-root DIR] [--force]
```

The script locates the pipeline root (explicit flag, environment variable `COVTL_PIPELINE_ROOT`, `./covtl-pipeline`, or a walk-up search), then performs three guarded steps. First, it copies the three module files to their destinations, byte-compiling each:

- `patch/files/core_morphing.py` → `vtl_synth/core/morphing.py`;
- `patch/files/cli_morph_cmd.py` → `vtl_synth/cli/morph_cmd.py`;
- `patch/files/test_morphing.py` → `tests/test_morphing.py`.

Second, it patches `vtl_synth/cli/main.py`: the anchor line

```
    # ---- global options (before the subcommand on the command line) ----
```

must occur exactly once — the script asserts it and aborts without writing otherwise — and the insertion of Section 1 is placed just before it, inheriting the file's line endings (the shipped `main.py` uses CRLF; the insertion is EOL-safe in both directions). The original file is backed up to `main.py.lambda-morph.bak`. Third, it verifies the result by byte-compiling `main.py` and probing, in a subprocess, that `build_parser()` now exposes the `morph` subcommand. The applied insertion is exactly:

```
    # ---- morph (lambda-morph option, covtl-lambda-patch) ---------------
    # 2026-10-22: articulatory interpolation continuum option;
    # 2026-10-26: v1.2.0 semantic -- the lambda variation acts on
    # the tautosyllabic CC cluster C1C2 -> C1'C2: --targets moves
    # the C1 polar target (theta, optionally rho) toward the C2
    # target; the two-endpoint tract blend remains available.
    # Additive option -- no existing subcommand is modified.
    # Implementation: vtl_synth/core/morphing.py
    # + vtl_synth/cli/morph_cmd.py; tests: tests/test_morphing.py.
    # Apply/revert: patch/apply_patch.py, patch/revert_patch.py.
    from vtl_synth.cli.morph_cmd import add_morph_subparser
    add_morph_subparser(sub, _add_engine_options)
```

The script is idempotent: re-running it on a patched tree exits with code 2 and a clear message (`--force` re-copies the module files; the `main.py` insertion stays guarded by its own marker).

3.3 Removing: `patch/revert_patch.py`

```
python patch/revert_patch.py [--pipeline-root DIR] [--keep-modules]
```

Reversion tries two strategies in order: restore `main.py` from the backup written at apply time; or, if the backup is gone, remove the insertion block by its comment markers with an EOL-agnostic regular expression. It then deletes the three added files (unless `--keep-modules` is given, for users who want to

keep calling the Python API), byte-compiles `main.py` and reports. Other engine patches — for instance the `/N/` closure patch in `constants.py` — are never touched. After a revert, the SHA-256 of `main.py` is byte-identical to its pre-patch value (verified during the release tests).

3.4 Non-regression statement

The patch is *additive*. It adds three files and one guarded insertion that only registers a new subcommand; no existing code path, default, file format or output changes. The engine's regression baselines (`regression_baselines*.json`) are unaffected, and `vtl-synth run`, `text-to-wav`, `inspect`, `plot-tract`, the polar and prosody options all behave exactly as before — importing `morphing.py` executes no engine code (it only imports constants).

4 Command-line reference

4.1 Synopsis

```
# target mode (DEFAULT semantic, v1.2.0): one cluster phrase
vtl-synth morph --phonetic --targets PHRASE [-o DIR] [--label NAME]
    [--lambdas LIST | --n-steps N] [--target-rho] [--cluster N]
    [--no-audio] [engine options]

# blend mode (option): two endpoints
vtl-synth morph A B [-o DIR] [--label NAME]
    [--lambdas LIST | --n-steps N] [--glottis {common,interpolate}]
    [--tol TOL] [--confine T1:T2 | --confine-frames F1:F2] [--ramp N]
    [--no-audio] [--text] [engine options]
```

In the target mode `PHRASE` is the *canonical cluster* phrase in engine SAMPA, dotless (`klinpan`, `klinkan`, `minbot` ...): the option locates the nasal-initial CC cluster (its C1 and C2 polar targets are read from the engine's own syllabic build), interpolates the C1 target per λ and re-synthesizes. A phrase carrying a syllable dot is rejected (C.C splits are out of scope, Section 2.1).

In the blend mode `A` is the canonical variant and `B` the assimilated one. Each argument is either a `.tract` file (file mode, when the name ends with `.tract`) or a phrase (phrase mode); the two must be of the same kind. In phrase mode the inputs are engine SAMPA by default — `klinpan`, `klimpan` — and the option synthesizes both endpoints first, with *identical* engine settings, in `<out>/_endpoints/`; pass `--text` to run English text through the g2p instead (assimilated spellings like *cleam* are usually not in the lexicon, and g2p output carries syllable dots, so SAMPA is the recommended input for the lambda variation).

Table 1 lists the options; the engine options (`-s`, `-l`, `--tcons`, …) are the standard ones of `vtl-synth run` and apply to the synthesis of the target-mode continuum (or of the two blend-mode endpoints).

4.2 A worked session

```
$ vtl-synth morph --phonetic --targets klinpan -o out/P1 --label P1
[morph] mode      : targets (C1C2 -> C1'C2 on the tautosyllabic cluster --
    DEFAULT semantic, v1.2.0)
[morph] label     : P1
[morph] frames    : 584 @ 400 Hz
[morph] cluster   : C1 n (rho 1.150, theta 4.7124 rad) -> C2 p (rho 1.060, theta
    0.8727 rad); dtheta +140.00 deg, theta only
[morph] C1 window : frames 232-271 (phoneme span); C1-incident arcs frames 232-311
[morph] footprint : frames 229-307 (measured, all lambdas vs canon)
[morph] check     : lam0_bitwise_canon = True
[morph] check     : glottis_bitwise_constant = True
[morph] check     : lam1_theta_equals_c2 = True
```

Table 1: Options of the `morph` subcommand.

Option	Default	Description
`-o`, `--output-dir`	`morph_out`	destination of the continuum files and of `morph_meta.json`.
`--label`	derived	file stem of the items (`<label>_l<lam>.tract`); default `<A>-vs-<B>` in blend mode, the phrase in target mode.
`--lambdas`	—	comma-separated weights in $[0, 1]$, e.g. `0,0.25,0.5,0.75,1`; deduplicated, sorted.
`--n-steps`	9	uniform grid of $N \geq 2$ steps from 0 to 1 inclusive.
`--targets`	off	**default semantic** (v1.2.0): single-phrase target morph on the CC cluster (Section 2.1); `B` must be absent.
`--target-rho`	off	target mode: also interpolate the C1 rho toward the C2 rho (default: theta only).
`--cluster`	1	target mode: 1-based rank of the nasal-initial CC cluster when the phrase carries several.
`--glottis`	`common`	blend-mode glottis policy of Section 2.4 (target mode: bitwise constant by construction).
`--tol`	`1e-6`	tolerance of the `common` glottis identity check.
`--confine` / `--confine-frames`	whole sequence	blend mode: restrict the blend to the C1 window (seconds or 400 Hz frames); context held canonical at every λ.
`--ramp`	10	raised-cosine edge ramp of the confine window (frames).
`--no-audio`	off	write the `.tract` continuum only.
`--text`	off	blend-mode phrase inputs are English text (g2p) instead of SAMPA.

```
[morph] check     : outside_c1_arcs_bitwise_canon = True
[morph] lambda=0  theta_C1=+4.712389 rad  out/P1/P1_l0.tract  + out/P1/P1_l0.wav
[morph] lambda=0.125  theta_C1=+5.017822 rad  out/P1/P1_l0.125.tract  +
    out/P1/P1_l0.125.wav
    (seven further rows, one per lambda of the grid, omitted)
[morph] lambda=1  theta_C1=+0.872665 rad  out/P1/P1_l1.tract  + out/P1/P1_l1.wav
[morph] meta      : out/P1/morph_meta.json
```

The four `check` lines are the guarantees of Section 2.5; the `footprint` line is the measured divergence extent of all λ against the canonical cluster, bounded by the C1-incident arcs. A dotted phrase fails early with an explanatory error, as does a phrase without a nasal-initial cluster.

5 Python API reference

All entry points live in `vtl_synth.core.morphing`. Two exceptions signal misuse: `MorphError` (itself a `ValueError`) and its subclass `MorphAlignmentError`, raised on frame-count and layout mismatches.

5.1 Driving a continuum from Python

```python
from vtl_synth.core.morphing import morph_targets, morph_pair

# the DEFAULT semantic (v1.2.0): one cluster phrase, polar targets
meta = morph_targets(
    'klinpan',                                  # canonical cluster
    output_dir='work/lambda/P1',
    lambdas=[0.0, 0.125, 0.25, 0.375, 0.5, 0.625, 0.75, 0.875, 1.0],
```

Table 2: Public API of `vtl_synth.core.morphing`.

Symbol	Role
`morph_targets(phrase, output_dir, ...)`	driver (DEFAULT semantic): cluster phrase → continuum of re-synthesized `.tract` (+ `.wav`) + `morph_meta.json`.
`build_target_continuum(phrase, ...)`	arrays: per-λ rebuilt cluster (tract + glottis), plan, checks, footprint.
`plan_target_cluster(phrase, ...)`	locates the C1C2 cluster: C1/C2 polar targets, $\Delta\theta$, C1 windows; rejects dotted phrases.
`find_nasal_c1_clusters(nodes)`	nasal-initial tautosyllabic CC clusters of a syltraj node list.
`interp_target_theta/rho(th1, th2, lam)`	the target rule (1) with exact (verbatim) endpoints; `wrap_angle_delta` is the shortest-path difference.
`morph_pair(canon, assim, output_dir, ...)`	blend-mode driver: files → continuum of `.tract` (+ `.wav`) + `morph_meta.json`.
`build_continuum(A, B, lambdas, ...)`	blend-mode arrays → `ContinuumResult` (items, `AlignmentReport`, checks).
`interpolate_tract(A, B, lam)`	the raw blend rule (2) with shape and range guards.
`check_alignment(A, B)`	shapes/glottis/divergence-window report; raises on misalignment.
`confine_weight(n, first, last, ramp)`	raised-cosine window of the blend-mode confine.
`parse_lambda_grid(spec, n_steps)`	grid parsing/validation ($[0, 1]$, dedup, sort).
`read_tract_sequence(path)`	`.tract` → `(data, sr)`; `write_morph_tract(path, data, sr)` writes the canonical VTL header.
`window_state(data, t0, t1, mode)`	articulatory ground truth in a time window (`mid` frame or `mean`).
`MORPH_GLOTTIS_MODES`, `DEFAULT_N_STEPS`, `DEFAULT_GLOTTIS_TOL`, `NASAL_C1_KEYS`	module constants.

```
    rho_enabled=False,                          # or True: theta + rho
    label='P1')
print(meta['targets'])   # C1/C2 targets, dtheta, C1 windows
print(meta['checks'])    # lam0 canon, lam1 theta == C2, glottis,
                         # outside-C1-arcs canonical (all True)

# option: the two-endpoint tract blend (v1.0/v1.1 semantics)
meta = morph_pair(
    'work/base/P1_canon/P1_canon.tract',   # canonical endpoint
    'work/base/P1_assim/P1_assim.tract',    # assimilated endpoint
    output_dir='work/lambda/P1',
    lambdas=[0.0, 0.125, 0.25, 0.375, 0.5, 0.625, 0.75, 0.875, 1.0],
    glottis_mode='common',
    confine=(232, 272),
    label='P1')
```

Existing `.wav` files are not re-synthesized unless `overwrite=True` is passed, so a failed downstream stage can resume where the synthesis stopped — the same convention as the kit experiment the option comes from.

5.2 The `morph_meta.json` record

The record is deliberately self-contained: together with the two hashed sources it is enough to re-derive every continuum file, and to audit that a reported stimulus set really is the convex combination it claims to be.

Table 3: Schema of `morph_meta.json` (target mode). The blend mode of v1.0/v1.1 replaces `targets`, `engine` and `diff_extent_frames` with `sources`, `glottis_mode`, `alignment` and `confine`.

Key	Content
`patch`, `mode`	patch identity string; `targets` (or `blend`).
`label`, `phrase`	item stem; the canonical cluster phrase (normalized engine input).
`semantic`	one-line statement of the v1.2.0 semantic.
`targets`	C1/C2 keys and polar targets, $\Delta\theta$ (rad and deg), `rho_enabled`, the C1 phoneme window and the C1-incident-arcs window (400 Hz frames).
`engine`	(co)articulation parameters actually used (speaker, f0, gesture timings, orthogonal branch).
`lambdas`	the grid, as floats.
`diff_extent_frames`	measured footprint of all λ against the canonical cluster.
`checks`	the guarantee booleans of Section 2.5.
`items`	one record per λ: weight, the λ-interpolated C1 target (`theta_c1`, optionally `rho_c1`), `.tract` path, `.wav` path (or `null`), frame count.

Table 4: The reference pairs (tautosyllabic clusters, dotless). *Licensed* refers to the phonological licensing of the assimilation: place agreement with the following stop is licit before a labial, illicit before an alveolar. The assimilated endpoint is not a second phrase: at $\lambda = 1$ the C1 polar target has reached the C2 target.

Pair	Canonical cluster	C2	Licensing	Transition
P1	`klinpan`	/p/	licensed	$\theta_{C1} : 270° \to 50°$ $(+140°)$
NV1	`klintan`	/t/	unlicensed	$\theta_{C1} : 270° \to -75°$ $(+15°)$

6 The assimilation continuum, end to end

6.1 The reference experiment

The example shipped with the patch (`examples/lambda_continuum.py`) reproduces the synthesis side of the original lambda experiment under the v1.2.0 target semantic. Two cluster pairs are built from engine SAMPA with default settings (speaker jd3, 100/80/200 ms timings):

Each pair is built on the nine-step grid $\{0, 0.125, \ldots, 1\}$; the articulatory ground truth is read at the C1 target window 0.58–0.68 s (mid frame 0.63 s) with `window_state`. Running the example takes a few minutes and prints, per pair, the cluster targets, the guarantee checks and the articulator table of Table 5.

The table reads as the anatomy of the target morph at mid-arc. The velum stays wide open (VO constant at 0.886): the continuum remains a nasal throughout — C1 keeps its nasal identity (selector, nasality envelope) while its *target* rotates toward the C2 place. At the mid-arc probe the place change is carried by the jaw and the tongue (JA falls by about one degree; TTX/TTY move monotonically, cf. Figure 2), while the lips still follow the vowel background (LP/LD constant to three decimals at this frame; the labial closure emerges along the second half of the arc). Unlike the blend mode these values are *not* linear in λ: they follow the engine's arc geometry (the raised-cosine mixing of departure and arrival targets), which is exactly the point — the continuum is a family of re-synthesized clusters, not a convex combination of two recordings.

6.2 Reading the figure

Figure 2 shows the same data as time series and as ground-truth curves.

Panel (a) makes the locality of the manipulation visible: the five JA trajectories share their onset and offset gestures and fan apart only inside the C1-incident arcs (the arrival arc into the nasal and the nasal→/p/ arc), re-merging before the following vowel; the TTX curves mirror them. The shaded band marks where downstream analyses should read the articulatory state. Panel (b) shows the arc-geometry

Table 5: Articulatory ground truth along the continuum (pair P1, $t = 0.63$ s, mid of the C1 arrival arc): lip proximity LP, lip distance LD, jaw angle JA (degrees) and velum opening VO, in engine units.

λ	LP	LD	JA	VO
0	0.177	0.645	−2.321	0.886
0.125	0.177	0.645	−2.181	0.886
0.25	0.177	0.645	−2.126	0.886
0.375	0.177	0.645	−2.162	0.886
0.5	0.177	0.645	−2.285	0.886
0.625	0.177	0.645	−2.484	0.886
0.75	0.177	0.645	−2.740	0.886
0.875	0.177	0.645	−3.030	0.886
1	0.177	0.645	−3.327	0.886

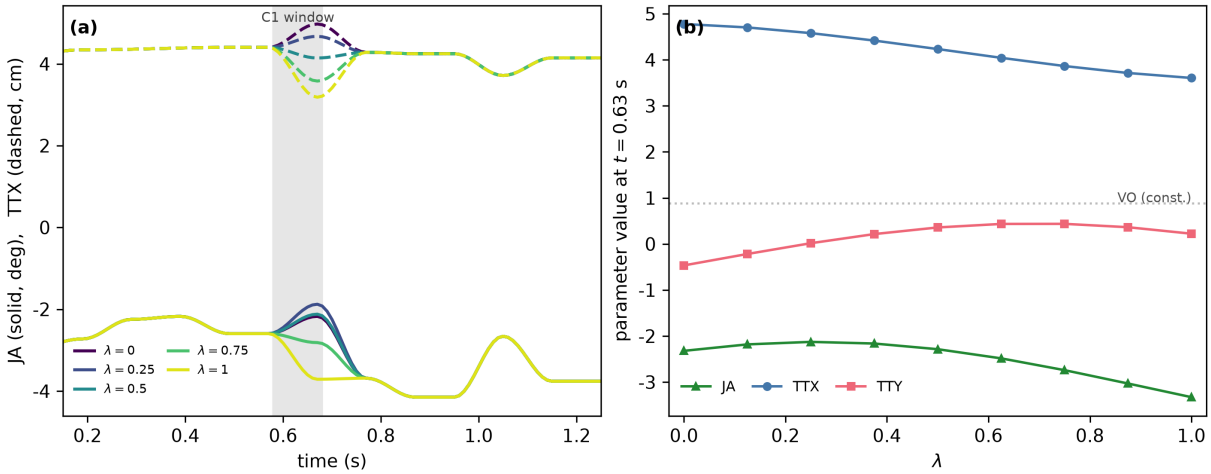


Figure 2: The P1 target continuum. (a) The articulators that carry the place change — jaw angle JA (solid) and tongue-tip x TTX (dashed) — over time for five weights of the grid (viridis ramp from $\lambda = 0$ dark purple to $\lambda = 1$ yellow); the shaded band is the 0.58–0.68 s C1 window. Outside the C1-incident arcs all curves superpose bitwise: the context (C2 /p/ included) never moves. (b) Articulatory ground truth at $t = 0.63$ s against λ: JA, TTX, TTY follow the arc geometry; VO is constant (dotted).

response of the three moving articulators and the flat nasality line — a pure place-of-articulation dial with the nasality valve left alone.

6.3 Relation to the source experiment

The option generalizes the kit's stage A lambda experiment, with one semantic correction: the kit synthesized *dotted* forms (`klin.pan`/`klim.pan`, C.C splits) and blended the two endpoint tracts; the v1.2.0 semantic operates on the *tautosyllabic cluster* (`klinpan`) and its polar targets, which is the object the assimilation process actually acts on (Section 2.1). The dotted blend remains reproducible with the blend mode on any frame-aligned pair, but it is no longer the documented semantic of the lambda variation; the kit's recorded dotted-stage values (LP $0.214 \rightarrow 0.742$, divergence frames 97–307) belong to that legacy path. The cluster example serves as the regression anchor of the target semantic: any future change in the engine's trajectory machinery that shifts the continuum (or the C1 windows) will surface here first.

7 Testing and guarantees

The patch ships `tests/test_morphing.py` — 53 tests in three layers, runnable from the pipeline root with `python -m pytest tests/test_morphing.py -q`. The pure-numpy layer (no Vocal-

Table 6: Test inventory (53 tests).

Layer	Count	What is pinned
Grids	8	default 9 steps; spec parse/dedup; N-step grids; range and format rejections (parametrized); $N \geq 2$.
Blend identities	4	bitwise $\lambda = 0/\lambda = 1$; exact midpoint; shape-mismatch and range guards.
Alignment + glottis	7	report fields and divergence window; frame-count mismatch; `common` rejection / `interpolate` blending; checks recorded; unknown mode; mode constants.
I/O	1	`.tract` round trip (header, 6 decimals).
Ground truth	3	mid-frame selection; window mean; mode guard.
Blend driver	3	continuum + meta on disk; grid-argument exclusivity; default label and grid.
Confine (v1.1.0)	7	weight shape/bounds/rejection/clamping; context-canonical guarantee; hard window at ramp 0; legacy unconfined behaviour; confine recorded in meta.
CLI parser	2	blend wiring; `--targets`/`--target-rho`/`--cluster` flags, single-phrase invocation.
Blend VTL	2	file-mode subcommand end to end; endpoint WAV determinism.
Target arithmetic	4	angle wrap; verbatim theta endpoints; verbatim rho endpoints; nasal key set.
Target planning	6	nasal clusters on the six Pouw pairs; dotted forms are not clusters; plan semantics + windows; dotted rejection; clusterless rejection; engine-override window shift.
Target synthesis (VTL)	4	continuum guarantees; rho channel; driver + meta; CLI end to end.
Target CLI rejections	2	dotted phrase; two-input misuse.

TractLab required) covers the blend interpolation identities (bitwise endpoints, exact midpoint), the grid parser (specs, uniform steps, out-of-range and malformed rejections), the alignment report (divergence window, glottis reporting), the glottis policies (rejection under `common`, blending under `interpolate`), the `.tract` write/read round trip with its six-decimal quantization, the `window_state` helper, the `morph_pair` driver without audio (file naming, metadata schema, mutual exclusion of grid arguments, confine records), the CLI parser wiring, and the pure target arithmetic (shortest-path angle wrap, verbatim endpoint identity of `interp_target_theta`/`interp_target_rho`). The engine-core layer (syltraj available, no VTL wheel) plans real cluster phrases: nasal-cluster detection on the six Pouw pairs (onset clusters correctly skipped), dotted-phrase rejection, C1/C2 target values, C1 windows and their engine-override shift. The VTL-gated layer synthesizes end to end: the target continuum guarantees of Section 2.5 on `klinpan`/`klinkan` (including the rho channel), the `morph_targets` driver and metadata, the blend-mode subcommand, and the determinism pin: the `.wav` of $\lambda = 0$ is byte-identical to a direct `synthesize_audio` of the canonical `.tract` content, while genuinely differing from the $\lambda = 1$ output.

8 Limitations and extensions

Three directions delimit the current design and sketch its natural extensions. First, the blend mode's frame-alignment contract excludes pairs that differ in segmental content — one cannot blend *clean pan* into *cleanly pan* without an upstream alignment stage; the target mode has no such constraint (one phrase in), but it is defined on nasal-initial CC clusters only, and on the two target channels θ (always) and ρ (optionally) — interpolating further properties of the C1 gesture (selector set, nasality envelope) would be a semantic extension, deliberately not offered. Second, the target interpolation is linear in the polar angle along the shortest arc; exotic contexts where the articulatorily natural route is the long way around would need an orientation option. Third, the option currently outputs `.tract`/`.wav` pairs; feeding the continuum directly into the video or probing layers is left to composition — the metadata record (which now carries the per-item C1 target) is designed to be the glue.

References

[1] C. Pouw, M. de Heer Kloots, A. Alishahi, and W. Zuidema. Perception of phonological assimilation by neural speech recognition models. *Computational Linguistics*, 50(4):1557–1585, 2024.

[2] F. Berthommier, "A mathematical model of the vowel space," arXiv:2111.00868 [eess.AS], 2021.

[3] F. Berthommier, "Why can big.bi be changed to bi.gbi? A mathematical model of syllabification and articulatory synthesis," arXiv:2307.02299 [eess.AS], 2023.

[4] F. Berthommier, "Synthèse de syllabes avec un modèle de Maeda piloté par une représentation complexe," in *Actes des 35èmes Journées d'Études sur la Parole (JEP)*, pp. 541–550, ATALA/AFPC, 2024.

[5] P. Birkholz, "Modeling consonant-vowel coarticulation for articulatory speech synthesis," *PLoS ONE*, 8(4):e60603, 2013. VocalTractLab 2.4: https://www.vocaltractlab.de.

[6] M. G. Gaskell and W. D. Marslen-Wilson. Phonological variation and inference in lexical access. *Journal of Experimental Psychology: Human Perception and Performance*, 22(2):327–342, 1996.

[7] P. Krug, VocalTractLab-Python (vocaltractlab-cython). https://github.com/paul-krug/VocalTractLab-Python.

[8] covtl-lambda-patch contributors, *nasal_synthesis reproducibility kit* (internal), 2026: the wav2vec2 probing and lambda-continuum study this option was extracted from.